ALGORITHMS AND COMPLEXITY

1. a) For each of the following statements, state whether it is true or false and provide a supporting proof.

i) $n(n+1)(n+2) = O(n^4)$. [3]

Answer:
True. $n(n+1)(n+2)/n^4 \to 0$ *as* $n \to \infty$.

Majority got this right, but a common mistake was to write that it was $O(n^3)$ *and therefore couldn't be* $O(n^4)$.

ii) For every function $f(n) > 0$ we have $f(n) = \Theta(f(n))$. [3]

Answer:
True. For $n \geq 1$, $0 < 1 \cdot f(n) \leq f(n) \leq 1 \cdot f(n)$ *so take* $n_0 = 1, c_1 = c_2 = 1$.

Majority got this right, but not everyone could explain why $c_1 = c_2 = 1$.

iii) $e^{3n} = O(e^{2n})$. [3]

Answer:
False. $e^{3n}/e^{2n} = e^n \to \infty$ *as* $n \to \infty$.

Quite a few mistakes on this one. Most common was to say that $e^{3n} \geq e^{2n}$ *which isn't sufficient to disprove the statement because it ignores the constant.*

iv) $5^n/(2^n + 3^n) = \Theta(2^n)$. [3]

Answer:
True. $(5^n/(2^n+3^n))/2^n = 5^n(4^n+5^n)$. *Since denominator* $4^n+5^n \leq 5^n$ *clearly have the ratio is* ≤ 1. *Since* $4^n \leq 5^n$ *have* $4^n + 5^n \leq 2 \cdot 5^n$ *so ratio is* $\geq 1/2$.

Note that there were several variants of this question in the exam. Quite a few people found these difficult for a variety of different reasons (often algebraic mistakes).

b) Give a tight bound for each of the following recurrence relations, or explain why it is not possible to do so.

i) $T(n) = \sqrt{2}T(n/2) + \sqrt{n}$. [2]

Answer:
$a = \sqrt{2}, b = 2, d = 1/2$ *so* $\log_b a = \log_2 \sqrt{2} = 1/2 = d$ *so* $T(n) =$

Answers

$O(n^d \log n) = O(\sqrt{n} \log n)$.

Almost everyone got this (and variants) correct.

ii) $T(n) = 3T(n/2) + n^2$. [2]

Answer:
$a = 3, b = 2, d = 2$ *so* $\log_b a = \log_2 3 < 2 = d$ *so* $T(n) = O(n^2)$.

Almost everyone got this (and variants) correct.

iii) $T(n) = 4T(n/2) + \pi(n)$, where $\pi(n)$ is the number of prime numbers less than n. [4]

Answer:
$\pi(n) \leq n$ *so can write* $T(n) = 4T(n/2) + O(n)$. *So* $a = 4, b = 2, d = 1$, $\log_b a = \log_2 4 = 2 > 1 = d$ *so* $T(n) = O(n^2)$.

Most people got this right, but a very common mistake was to include the time taken to compute the number of primes less than n, which is not relevant.

Master Theorem. If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) + O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

Answers

2. The *closest pair problem* is as follows. You are given a set of points in a plane $\mathbf{z}_i = (x_i, y_i)$ where $i = 1$ to n. The distance between two points with indices i and j is given by

$$d_{ij} = \|\mathbf{z}_i - \mathbf{z}_j\| = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

You need to find the distance between the closest pair of points,

$$d_{\min} = \min_{i \neq j} d_{ij}.$$

a) The following pseudocode is a brute-force algorithm to solve this problem.

```
1:  function ClosestPairBruteForce({z_i}, n)
2:      d_min = ∞
3:      for i = 1 to n do
4:          for j = 1 to n do
5:              if i ≠ j then
6:                  d = ||z_i − z_j||
7:                  if d < d_min then
8:                      d_min = d
9:                  end if
10:             end if
11:         end for
12:     end for
13:     return d_min
14: end function
```

i) Briefly summarise the operation of the algorithm in one or two English sentences. [2]

Answer:
The algorithm considers all pairs of indices i and j with $i \neq j$, computes the distance between the two points, and keeps a running track of the smallest distance found while it runs.

Almost everyone was able to do this.

ii) What is the time complexity of the brute-force algorithm as a function of n? Justify your result. [3]

Answer:
Computing the distance is $O(1)$ and there are two nested loops each iterating over n items, so $O(n^2)$.

Almost everyone was able to do this.

iii) Suggest a modification of the algorithm, changing only one line to make it run almost twice as fast. [3]

Answer:
The innermost loop (line 4) can be changed to range from $j = i + 1$ to n because the distance metric is symmetric.

Almost everyone got this answer or a variant of it.

Answers

iv) Has this improved the time complexity of the improved version of the algorithm? [2]

Answer:
No. Still $O(n^2)$.

Almost everyone was able to do this.

Answers

b) Consider a divide and conquer approach to this problem, where we split the points up using their x-coordinates. This forms two equal subproblems, consisting of a "left" set and a "right" set of points.

Shown below is an incomplete pseudocode implementation of this algorithm, with missing lines indicated by "...".

The function MEDIAN($\{v_i\}$) returns the median of a set of values $\{v_i\}$, that is a number m such that half of the v_i satisfy $v_i \leq m$ and half satisfy $v_i > m$. The symbol x_i refers to the x-coordinate of the point $\mathbf{z}_i$.

```
1:  function ClosestPair({z_i}, n)
2:      if n ≤ 3 then
3:          return ClosestPairBruteForce({z_i}, n)
4:      end if
5:      m = Median({x_i for i = 1,...,n})
6:      L = {i for i = 1,...,n where x_i ≤ m}
7:      R = {j for j = 1,...,n where x_j > m}
8:      d^L_min = ClosestPair({z_i for i ∈ L}, Len(L))
9:      d^R_min = ClosestPair({z_j for j ∈ R}, Len(R))
10:     ...
11:     return d_min
12: end function
```

i) Briefly describe, and identify line numbers, for each of the following parts of the algorithm: (1) base case, (2) divide, (3) conquer. Note: the combine step is missing (line 10). [4]

Answer:
Base case is handled in lines 2-4, and uses the brute force algorithm when it cannot be divided into subproblems of size at least 2. Divide step is handled in lines 5-7, points are assigned to the left set L if their x-coordinate is less than the median x-coordinate, otherwise to the right set R. Conquer step is handled in lines 8-9 where the algorithm is called recursively on the left and right sets.

Almost everyone was able to do this, although some only gave the line numbers.

ii) We can complete the combine step using the following logic.

Let (i^*, j^*) be the indices of the closest pair of points. There are three possibilities: (1) both points i^* and j^* are in the set L, (2) both points are in the set R, or (3) one point is in L and one point is in R. If the closest pair matches case (1) or (2), then it will be found in one of the two subproblems. So, for the missing combine step, only case (3) needs to be checked.

Write pseudocode for the missing combine step that checks distances between all pairs (i, j) where $i \in L$ and $j \in R$, and finds the distance between the closest pair in the whole set. Inserting this pseudocode at line 10 should complete the partial algorithm above. [4]

Answer:
The following pseudocode implements this in a naive way:

Answers

$d_{\min} = \min\{d^L_{\min}, d^R_{\min}\}$
for $i \in L$ ***do***
  for $j \in R$ ***do***
    $d = \|\mathbf{z}_i - \mathbf{z}_j\|$
    if $d < d_{\min}$ ***then***
      $d_{\min} = d$
    end if
  end for
end for

The majority were able to write this code, although there was often confusion about indices.

iii) What is the time complexity of your code for the combine step? Justify your result. [2]

Answer:
There are two nested loops, each of size $n/2$ and the inner code is $O(1)$ so it is $O(n^2)$.

Most people were able to do this.

iv) What is the complexity of this divide and conquer algorithm? Justify your result. You may assume that the median function is $O(n)$. [2]

Answer:
There are two subproblems of size $n/2$ and the remainder of the code including the combine step is $O(n^2)$ (because the median is assumed to be $O(n)$). So if $T(n)$ is the running time then $T(n) = 2T(n/2) + O(n^2)$. Applying the master theorem with $a = b = d = 2$ we see $\log_b a = \log_2 2 = 1 < d = 2$ so $T(n) = O(n^d) = O(n^2)$.

Note that despite having the same theoretical complexity, even this naive divide and conquer algorithm is faster in practice than the improved brute force algorithm because you need to compute roughly half as many distances.

The majority were able to do this, although some peole did fail to apply the master theorem.

Answers

c) The time complexity of this algorithm can be improved by modifying the combine step.

i) If the combine step could be reduced to a time complexity of $O(n)$, what would be the time complexity of the divide and conquer algorithm? [2]

Answer:
We would now have $T(n) = 2T(n/2) + O(n)$ so $a = b = 2$ and $d = 1$ so $\log_b a = \log_2 2 = 1 = d$ and therefore applying the master theorem $T(n) = O(n^d \log n) = O(n \log n)$.

Almost everyone was able to do this, even if they hadn't done previous parts.

ii) The combine step from (b)(ii) checks the distance between pairs of points with one point on the left and one on the right. Given $w > 0$, consider a pair of points (i, j) with $i \in L$ and $j \in R$ with $x_i < m - w$ or $x_j > m + w$.

What is the smallest possible value for $\|\mathbf{z}_i - \mathbf{z}_j\|$?

Suggest a modification of the combine step to avoid checking distances for points that must be further apart than the closest pair found in the divide and conquer steps.

Compute the time complexity for your modified combine step. You do not need to write pseudocode. [3]

Answer:
If $x_i < m - w$ or $x_j > m + w$ then (given that in all cases $x_i \leq m$ and $x_j > m$) the distance $\|\mathbf{z}_i - \mathbf{z}_j\| > w$. If we set $w = d_{\min}$ then we can ignore points outside the strip $m - d_{\min} < x < m + d_{\min}$. It may be the case that all points are within this strip, and therefore the time complexity is unchanged, $O(n^2)$.

Very few people managed to get all the available marks for this part. A very common mistake was to give $2w$ as the minimum distance because they were assuming an and *rather than an* or. *Some people suggested an algorithm of checking all pairs (i, j) but only computing the distance if $|x_i - x_j| < d_{\min}$ which was OK although not exactly what the question was getting at.*

iii) By applying the same reasoning to the y-coordinate, suggest a further improvement to the combine step that reduces the time complexity of the combine step to $O(n)$.

You may assume that all points have been pre-sorted with increasing y-coordinate.

Hint: use the fact that within any square of side length $d_{\min}$ there can be at most 4 points in L or 4 points in R. [3]

Answer:
Given points (i, j) within the vertical strip, we only need to check the

Answers

distance if $|y_i - y_j| < d_{\min}$. Further, we can assume $y_i < y_j$ by symmetry, i.e. $i < j$ (since the points are sorted in ascending order of y-coordinate). Now, using the hint, we can see that if we iterate through the points in the vertical strip in order of increasing y-coordinate, we only need to check at most 7 points ahead, and so the combine step will only require at most $7n$ comparisons and is therefore $O(n)$.

Note that this makes the whole algorithm $O(n \log n)$ and therefore we can carry out the pre-sorting stage without changing the complexity. Note also that we can simultaneously sort on x and y and retain these sorted copies in order to have a simple $O(n)$ implementation of the median without having to use complicated $O(n)$ median algorithms.

For interest, the following Python code implements this algorithm (ignoring the complexity of the median algorithm, which is actually $O(n \log n)$ in the NumPy library).

```
def divide_and_conquer(x, y, sorted=False):
    n = len(x)
    # handle base case
    if n<=3:
        d_min, _ = brute_force(x, y)
        return d_min
    # presorting: note that we don't worry about median extraction
    if not sorted:
        I = argsort(y)
        x = x[I]
        y = y[I]
    # divide
    m = median(x) # this is inefficient but can be made efficient
    left_indices = x<=m
    right_indices = x>m
    # conquer
    d_min_left = divide_and_conquer(x[left_indices],
                                    y[left_indices],
                                    sorted=True)
    d_min_right = divide_and_conquer(x[right_indices],
                                     y[right_indices],
                                     sorted=True)
    # combine
    d_min = min(d_min_left, d_min_right)
    within_strip = logical_and(x>=m-d_min, x<=m+d_min)
    xw = x[within_strip]
    yw = y[within_strip]
    n_within = len(xw)
    for i in range(n_within):
        for j in range(i+1, min(i+8, n_within)):
            d = sqrt((xw[i]-xw[j])**2+(yw[i]-yw[j])**2)
            if d<d_min:
                d_min = d
    return d_min
```

Answers

Very few people managed to get this right, and most didn't even attempt it.

Answers